Figure 3: Example of an e-commerce website interacting with multiple microservices to get details about a product

Scaling a monolithic application is nothing but replicating a monolith on multiple servers. But scaling a microservices application is distributing different microservices across servers.

The advantages of microservices are:

1. Services are loosely coupled.
2. CI/CD process can be made easy.
3. Scaling is very simple and robust.
4. There's no need to maintain a common codebase.
5. Developers can develop and deploy the service independently.
6. It is simpler to understand the functionality of a service.
7. Since services are decentralised, faults can be detected easily.

The drawbacks are:

1. It's difficult to integrate an application when it contains a large number of services.
2. More transactions will be involved.
3. Increased system resource overhead. Each service communicates through APIs/remote calls. It consumes more network bandwidth and system memory resources.
4. API gateway, router or load balancer is needed to route the request from the client to the different microservices, and for them to respond back to the client.

However, the microservices architectural style brings about a drastic change in the application development and release process.

Choosing between REST and AMQP for implementing microservices

It is very important to decide how microservices should communicate with each other. JSON is the *de facto* data format for exchanging information between services. REpresentation State Transfer (REST) over Hyper Text Transfer Protocol (HTTP) or Advanced Message Queuing Protocol (AMQP) over Remote Procedure Call (RPC) can be used for communication.

There are several advantages and disadvantages of using either REST or AMQP. The security (whether synchronous or asynchronous) called between services has to be considered.

REST has several advantages like securing API endpoints with many third party authorisation plugins and can expose the API to the public. It is easy to document the REST API using tools like Swagger. The main drawback of using REST over HTTP is its blocking capacity. REST API communication is synchronous—while invoking a REST call one has to wait for the response from the server before processing another request. Here some actions are lost if a particular service is not reachable. The service is blocked and this impacts the performance of the application.

The performance of the microservices can be made efficient by using AMQP. Here the transactions are asynchronous. If a request is needed to be made to another service, a message can be sent on the topic and the next action can be performed immediately.

In an HTTP scenario, if a request is made and the callee is down, the connection will be refused and one will have to try again and again until it comes back up.

With message systems, requests can simply be sent and done with. If the callee cannot receive it, the broker will keep the message in the queue, and then deliver it when the callee connects back. The response will come back when it can and won't need to be blocked or waited upon.

How to choose platforms for implementing microservices

1. Choose the micro Linux platforms (lightweight operating systems in Linux) such as CoreOS, Atomic or Photon. These operating systems are optimised for running containers. There is no need to run full-featured operating systems like Ubuntu or Red Hat, since there is no need for all those functionalities that a typical OS should have. It will impact the system performance as well.
2. It is better to choose a container environment rather than a virtual machine. When it comes to open source container platforms, Docker is the best choice.
3. Swarm or Kubernetes can be used for scheduling and orchestrating container based applications.
4. For orchestrating deployments, popular frameworks like Red Hat's OpenShift may be used.
5. Prometheus is a good option for monitoring the applications.

The role of an API gateway in microservices

An API gateway is the single entry point for all clients. It handles requests in one of two ways. Some requests are simply proxied or routed to the appropriate service. It handles other requests by fanning out to multiple services. There are a few open source and proprietary API gateways that are available in the market for implementing the microservices based architecture. Besides, the IaaS cloud providers have created a platform for managing APIs for microservices. An example is the AWS API Gateway.